

SwiftHaul Logistics Analysis

Rodolfo Yoshii

GitHub: https://github.com/taloc/SwiftHaul_Logistics_Analysis

The SwiftHaul Logistics analysis is data analysis of a fictitious company, where the idea is to demonstrate the usage of Google's BigQuery and Cloud Storage.

The below summary provides the high-level results from the analysis of carrier reliability, and costs. Detailed data ingestion, storage, and the setup of dataset, tables and queries can be found in the github repository.

Appendix: Query optimization screenshots

Data governance and access

SwiftHaul relies on Google's data governance for encryption of at rest and during transit, ensuring that our data is secured from end-to-end before the data is stored, and while we operate BigQuery for internal and external transmission of data through the internet.

For this current exercise we have IAM full access control at all levels of project, dataset, tables, and views. Permissions could be customized for limited access as needed.

Carrier reliability

The three most reliable carriers by on-time delivery are **Skyliner Air**, **SwiftHaul Ground**, and **MetroCourier**. The three least reliable are **OceanLink**, **BlueLine Freight**, and **Continental Cargo**.

carrier	total_shipments	on_time_shipmen...	on_time_rate_pct
Skyliner Air	250025	238006	95.19
SwiftHaul Ground	250269	233183	93.17
MetroCourier	249964	230329	92.14
Continental Cargo	249989	222805	89.13
BlueLine Freight	249669	217822	87.24
OceanLink	250084	203021	81.18

Lanes cost analysis

The 3 most expensive lanes by average freight-cost are:

APAC-East -> US-Central,

US-West -> APAC-South,

EU-West -> APAC-South, with an average cost of \$403.31, \$402.96, and \$402.85 respectively.

origin	destination	lane	average_cost	shipments_per_la...
APAC-East	US-Central	APAC-East -> US-Central	403.31	26719
US-West	APAC-South	US-West -> APAC-South	402.96	26706
EU-West	APAC-South	EU-West -> APAC-South	402.85	26581
APAC-East	EU-Central	APAC-East -> EU-Central	402.41	26983
US-East	APAC-East	US-East -> APAC-East	402.2	26873
US-Central	EU-Central	US-Central -> EU-Central	402.12	26637
EU-West	US-South	EU-West -> US-South	401.95	26778
US-South	EU-Central	US-South -> EU-Central	401.87	26667
US-East	EU-Central	US-East -> EU-Central	401.83	26892

SwiftHaul Logistics Analysis

Recommendations: How do carriers compare on cost versus reliability

Carrier choice is a trade-off between cost and on-time performance. The analysis starts with the three busiest lanes, then selects carriers on those lanes with on-time performance of 90% or higher, ranked by average cost.

Top 3 busiest lanes:

Lane	Number of shipments
EU-Central -> APAC-South	27184
EU-West -> APAC-East	27081
EU-West -> EU-Central	27034

Identification of top carrier per lane based first on average cost and filtered for only those with 90%+ on-time performance.

Lane	Carrier	Average cost	On-time rate
EU-Central -> APAC-South	SwiftHaul Ground	\$249.54	93.01%
EU-West -> APAC-East	SwiftHaul Ground	\$247.89	93.01%
EU-West -> EU-Central	SwiftHaul Ground	\$155.05	92.99%

Conclusion:

SwiftHaul Ground is the lowest-cost option on all three lanes and stays above our internal 90% on-time threshold. That result should be read with care: ground freight is not the same service as air. Among air and courier providers, the next-best options are MetroCourier and Skyliner Air.

SwiftHaul Logistics Analysis

Appendix:

```
-- Unoptimized query takes 189.03 MB when run.
SELECT
  *
FROM `lab16499377-1788299418.swift haul.shipments`;
```

The screenshot shows a query execution interface with a menu bar (File, Edit, Run, Tools) and a toolbar (Run, Save, Schedule, Share). The query editor contains the following SQL:

```
1 -- Unoptimized query takes 189.03 MB when run.
2 SELECT
3   *
4 FROM `lab16499377-1788299418.swift haul.shipments`;
```

Below the editor, a message states: "This query will process 189.03 MB when run." and "Using on-demand processing quota".

The interface shows a list of recent queries on the left, with the current query selected. The main panel displays the "Query results" tab, which includes a "Job information" section with the following details:

Job information	Results	Visualization	JSON	Execution details	Execution graph
For help debugging or optimizing your query, check our documentation. Learn more					
Elapsed time	Slot time consumed	Bytes shuffled	Bytes spilled to disk		
5 sec 234 ms	9 sec 115 ms	578.52 MB	0 B		

```
-- Optimized query used for getting average freight cost per lane of delayed shipments
processes in 35.29MB
SELECT
  carrier,
  DATE_TRUNC(ship_date, MONTH) AS ship_month,
  COUNT(*) AS total_shipments,
  COUNTIF(on_time = TRUE) AS on_time_shipments,
  ROUND(
    100 * COUNTIF(on_time = TRUE) / COUNT(*),
    2
  ) AS on_time_rate_pct
FROM `lab16499377-1788299418.swift haul.shipments`
WHERE ship_date IS NOT NULL
GROUP BY
  ship_month,
  carrier
ORDER BY
  carrier,
  ship_month
;
```

SwiftHaul Logistics Analysis

```
6 -- Optimized query used for getting average freight cost per lane of delayed shipments processes in 35.29MB
7 SELECT
8     carrier,
9     DATE_TRUNC(ship_date, MONTH) AS ship_month,
10    COUNT(*) AS total_shipments,
11    COUNTIF(on_time = TRUE) AS on_time_shipments,
12    ROUND(
13        100 * COUNTIF(on_time = TRUE) / COUNT(*),
14        2
15    ) AS on_time_rate_pct
16 FROM `lab16499377-1788299418.swifthaul.shipments`
17 WHERE ship_date IS NOT NULL
18 GROUP BY
19     ship_month,
20     carrier
21 ORDER BY
22     carrier,
23     ship_month
24 ;
25
```

This query will process 35.29 MB when run.

Using on-demand processing quota

Recent

Query results

Chat View all jobs Save results Open in

10:00 PM

Job information Results Visualization JSON Execution details Execution graph

Optimized quer...

9:58 PM

SELECT * FROM `L

9:37 PM

SELECT origin_re...

Elapsed time	Slot time consumed	Bytes shuffled	Bytes spilled to disk
396 ms	117 ms	7.42 KB	0 B

```
-- Creating optimized shipments table with partition over ship_date and cluster by
carrier and destination region
CREATE OR REPLACE TABLE `lab16499377-1788299418.swifthaul.optimized_shipments`
PARTITION BY ship_date
CLUSTER BY carrier, destination_region AS
SELECT
    *
FROM `lab16499377-1788299418.swifthaul.shipments`;
```

```
SELECT
    carrier,
    DATE_TRUNC(ship_date, MONTH) AS ship_month,
    COUNT(*) AS total_shipments,
    COUNTIF(on_time = TRUE) AS on_time_shipments,
    ROUND(
        100 * COUNTIF(on_time = TRUE) / COUNT(*),
        2
    ) AS on_time_rate_pct
FROM `lab16499377-1788299418.swifthaul.shipments`
WHERE ship_date IS NOT NULL
GROUP BY
    ship_month,
    carrier
ORDER BY
    carrier,
    ship_month
```

SwiftHaul Logistics Analysis

```
34 -- Running the same filtered query but in an optimized table will process in 35.29 MB
35 SELECT
36     carrier,
37     DATE_TRUNC(ship_date, MONTH) AS ship_month,
38     COUNT(*) AS total_shipments,
39     COUNTIF(on_time = TRUE) AS on_time_shipments,
40     ROUND(
41         100 * COUNTIF(on_time = TRUE) / COUNT(*),
42         2
43     ) AS on_time_rate_pct
44 FROM `lab16499377-1788299418.swifthaul.shipments`
45 WHERE ship_date IS NOT NULL
46 GROUP BY
47     ship_month,
48     carrier
49 ORDER BY
50     carrier,
51     ship_month
52 ;
53
```

✓ This query will process 35.29 MB when run.

Using on-demand processing quota

Recent

Query results

Chat

View all jobs

Save results

Open in

10:02 PM

Job information

Results

Visualization

JSON

Execution details

Execution graph

✓ SELECT carrier, D...

10:00 PM

✓ -- Optimized quer...

9:58 PM

✓ SELECT * FROM 'L...

For help debugging or optimizing your query, check our documentation. [Learn more.](#)

Elapsed time	Slot time consumed	Bytes shuffled	Bytes spilled to disk
276 ms	139 ms	7.42 KB	0 B

```
-- Running the same filtered query but in an optimized table will process in 35.29MB
SELECT
    carrier,
    DATE_TRUNC(ship_date, MONTH) AS ship_month,
    COUNT(*) AS total_shipments,
    COUNTIF(on_time = TRUE) AS on_time_shipments,
    ROUND(
        100 * COUNTIF(on_time = TRUE) / COUNT(*),
        2
    ) AS on_time_rate_pct
FROM `lab16499377-1788299418.swifthaul.optimized_shipments`
WHERE ship_date IS NOT NULL
GROUP BY
    ship_month,
    carrier
ORDER BY
    carrier,
    ship_month
;
```

SwiftHaul Logistics Analysis

54 -- Running the same filtered query but in an optimized table will process in 35.29MB

55 SELECT

56 carrier,

57 DATE_TRUNC(ship_date, MONTH) AS ship_month,

58 COUNT(*) AS total_shipments,

59 COUNTIF(on_time = TRUE) AS on_time_shipments,

60 ROUND(

61 100 * COUNTIF(on_time = TRUE) / COUNT(*),

62 2

63) AS on_time_rate_pct

64 FROM 'lab16499377-1788299418.swift_haul.optimized_shipments'

65 WHERE ship_date IS NOT NULL

66 GROUP BY

67 ship_month,

68 carrier

69 ORDER BY

70 carrier,

71 ship_month

Query completed

Using on-demand processing quota

Recent

Query results

Chat

View all jobs

Save results

Open in

10:09 PM

Job information

Results

Visualization

JSON

Execution details

Execution graph

Running the sa...

10:02 PM

SELECT carrier, D...

10:00 PM

Optimized quer...

For help debugging or optimizing your query, check our documentation. [Learn more](#)

Elapsed time

Slot time consumed

Bytes shuffled

Bytes spilled to disk

1 sec 118 ms

18 sec 834 ms

194.2 KB

0 B